

矩阵的舞蹈

多维数组与矩阵模式 · 12题 · C++ & Python 双语对照

NQ058 数组的右上半部分 AcWing 745

输入一个二维数组 $M[12][12]$ ，根据输入的要求，求出二维数组的右上半部分元素的平均值或元素的和。右上半部分是指主对角线（如下图中黄色部分）上方的部分（绿色部分）。

输入 第一行输入一个大写字母，若为S，则表示需要求出右上半部分的元素的和，若为M，则表示需要求出右上半部分的元素的平均值。接下来 12 行，每行包含 12 个用空格隔开的浮点数，表示这个二维数组，其中第 $i + 1$ 行的第 $j + 1$ 个数表示数组元素 $M[i][j]$ 。

输出 输出一个数，表示所求的平均数或元素的和的值，保留一位小数。

来源 AcWing 745

输入	输出
M	-1.2
-6.5 8.2 0.7 9.0 8.4 -8.3 0.9 -0.7 -7.9	
7.1 -1.6 4.6	
-9.4 -9.0 1.5 -9.0 -5.1 -0.5 -2.8 -9.1	
8.0 -6.9 -5.5 -6.6	
-6.8 0.3 3.8 6.1 -9.9 -9.3 8.5 8.6 5.0	
6.9 -3.6 -3.0	
5.4 -5.2 5.0 -5.3 -7.9 -9.7 7.7 -8.2	
0.0 -4.7 -8.2 -9.2	
-0.8 -6.0 -5.4 -5.1 -7.9 -0.5 -5.3 -5.7	
-4.6 9.6 -8.2 -9.2	
7.8 -5.8 -5.8 -5.8 7.2 0.5 -7.9 1.2	
-6.8 -9.1 0.3 -1.4	
4.3 -7.2 3.5 -6.4 -9.1 -6.0 3.5 -5.1	
-5.6 -6.9 -9.1 -2.1	
-3.4 -3.8 -9.4 -7.1 -8.4 -9.9 -5.7 -8.3	
-5.3 -7.4 -8.5 -4.8	
-0.2 0.3 -4.2 8.4 0.7 -6.4 -3.7 3.5	
-0.9 3.7 0.9 -2.7	
7.1 0.1 -8.4 -0.1 -7.9 -4.4 -7.6 -2.7	
5.5 -0.9 -7.0 -2.0	
9.6 -9.8 3.3 -9.9 -6.8 6.7 3.1 1.2 -9.5	
-4.3 -1.7 -9.7	
1.8 5.5 -0.7 -0.9 3.2 2.5 0.5 7.3 8.3	
0.3 0.9	

提示 注意输出格式。

C++

```
#include <cstdio>

int main()
{
    char t;
    scanf("%c", &t);
    double a[12][12];

    for (int i = 0; i < 12; i ++ )
        for (int j = 0; j < 12; j ++ )
            scanf("%lf", &a[i][j]);

    int c = 0;
    double s = 0;

    for (int i = 0; i < 12; i ++ )
        for (int j = i + 1; j < 12; j ++ )
        {
            c ++ ;
            s += a[i][j];
        }

    if (t == 'S') printf("%.1lf\n", s);
    else printf("%.1lf\n", s / c);

    return 0;
}
```

Python

```
# AcWing 745
# Python solution
```

NQ059 数组的左上半部分 AcWing 747

输入一个二维数组 $M[12][12]$ ，根据输入的要求，求出二维数组的左上半部分元素的平均值或元素的和。左上半部分是指次对角线（如下图所示，黄色部分为次对角线，绿色部分为左上半部分）上方的部分。

输入

第一行输入一个大写字母，若为S，则表示需要求出左上半部分的元素的和，若为M，则表示需要求出左上半部分的元素的平均值。接下来 12 行，每行包含 12 个用空格隔开的浮点数，表示这个二维数组，其中第 $i + 1$ 行的第 $j + 1$ 个数表示数组元素 $M[i][j]$ 。

输出

输出一个数，表示所求的平均数或和的值，保留一位小数。

来源

AcWing 747

输入

```
M
-0.4 -7.7 8.8 1.9 -9.1 -8.8 4.4 -8.0
0.5 -5.8 1.3 -8.0
-1.7 -4.6 -7.0 4.7 9.6 2.0 8.2 -6.4 2.2
2.3 7.3 -0.4
-8.1 4.0 -6.9 8.1 6.2 2.5 -8.2 -6.2
-1.5 9.4 -9.8 -3.5
-3.4 -2.4 -8.0 0.7 -2.9 5.4 -8.4 -5.4
-5.1 -6.0 -8.5 8.1
6.2 -1.5 -6.9 3.1 -7.9 5.1 -8.8 0.9
-7.4 -3.9 -2.7 0.9
-6.8 -0.8 -9.9 9.1 -3.7 -8.4 4.4 9.8
```

输出

```
-0.8
```

```
-6.3 -6.4 -3.7 2.8
-3.8 -5.4 -4.6 2.0 4.0 9.2 -8.0 0.5
-3.9 6.5 -4.3 -9.9
-7.2 -6.2 -1.2 4.1 -7.4 -4.6 4.7 -0.4
-2.2 -9.1 0.4 -5.8
-8.0 -2.0 -5.3 -3.7 -2.2 -3.9 -8.6 -7.0
-4.1 -3.2 -5.9 -4.2
-1.2 -6.2 -8.2 7.0 -5.3 4.4 9.5 7.2 2.3
4.4 3.2 -2.0
-6.9 -6.2 5.1 8.5 7.1 -0.8 -0.7 2.7
-6.0 4.2 -8.2 -9.8
-3.5 -1.7 5.4 2.8 1.6 -1.0 -6.1 7.7
-6.5 -8.3 -8.5 9.4
```

提示 注意输出格式。

C++

```
#include <cstdio>

int main()
{
    double a[12][12];
    char t;

    scanf("%c", &t);
    for (int i = 0; i < 12; i ++ )
        for (int j = 0; j < 12; j ++ )
            scanf("%lf", &a[i][j]);

    int c = 0;
    double s = 0;
    for (int i = 0; i < 12; i ++ )
        for (int j = 0; j <= 10 - i; j ++ )
        {
            c ++ ;
            s += a[i][j];
        }

    if (t == 'S') printf("%.1lf\n", s);
    else printf("%.1lf\n", s / c);

    return 0;
}
```

Python

```
# AcWing 747
# Python solution
```

NQ060 数组的上方区域 AcWing 749

输入一个二维数组 $M[12][12]$ ，根据输入的要求，求出二维数组的上方区域元素的平均值或元素的和。数组的两条对角线将数组分为了上下左右四个部分，黄色部分为对角线，绿色部分为上方区域。

输入

第一行输入一个大写字母，若为S，则表示需要求出上方区域的元素的和，若为M，则表示需要求出上方区域的元素的平均值。接下来 12 行，每行包含 12 个用空格隔开的浮点数，表示这个二维数组，其中第 $i + 1$ 行的第 $j + 1$ 个数表示数组元素 $M[i][j]$ 。

输出

输出一个数，表示所求的平均数或和的值，保留一位小数。输出结果与标准答案核对误差不超过 0.1 即视为正确。

来源

AcWing 749

输入

```
S
-4.8 -8.0 -2.9 6.7 -7.0 2.6 6.5 1.7 1.9
5.6 -1.6 -6.3
-4.3 1.5 8.7 -0.3 5.4 -9.3 4.8 7.0 3.6
-8.3 -1.0 1.3
5.0 -4.9 2.7 -2.0 -1.7 -5.4 8.0 -7.3
2.3 -1.9 -4.0 2.9
2.8 -5.8 -0.0 2.2 4.8 7.7 -3.0 -7.5
-3.5 9.7 -4.3 -8.6
-1.8 -1.0 9.0 9.3 -3.7 -1.1 0.8 -0.2
-0.0 9.9 4.5 -0.7
3.8 -3.9 2.1 -9.7 5.5 9.4 -4.6 3.3 -9.6
5.1 -4.5 1.5
2.8 -5.9 -1.7 -7.4 5.0 -6.4 -5.1 -8.1
-5.2 -1.8 -2.8 -1.7
8.8 2.8 -4.1 7.1 8.4 -5.6 3.9 -9.7 -7.1
1.3 2.3 -3.3
1.7 5.1 0.1 9.2 4.5 9.7 7.2 8.6 8.7 1.1
6.7 0.3
-3.6 -7.1 -8.9 7.1 -5.9 1.6 -7.4 6.7
3.9 4.3 -2.4 -3.7
8.9 -6.2 5.0 -8.6 -1.3 -8.8 2.6 8.9 5.5
9.0 -2.2 -4.4
5.7 3.7 1.8 -2.1 -7.3 -7.9 4.7 6.0 3.3
-2.8 1.4 -6.9
```

输出

21.7

提示 注意输出格式。

C++

```
#include <cstdio>
#include <iostream>

using namespace std;

int main()
{
    char t;
    cin >> t;
    double m[12][12];

    for (int i = 0; i < 12; i++)
        for (int j = 0; j < 12; j++)
            cin >> m[i][j];

    double s = 0, c = 0;
    for (int i = 0; i < 5; i++)
        for (int j = i + 1; j <= 10 - i; j++)
        {
            c += 1;
            s += m[i][j];
        }
    if (t == 'M') printf("%.1lf\n", s / c);
    else printf("%.1lf\n", s);

    return 0;
}
```

Python

```
# AcWing 749
# Python solution
```

NQ061 数组的左方区域 AcWing 751

输入一个二维数组 $M[12][12]$ ，根据输入的要求，求出二维数组的左方区域元素的平均值或元素的和。数组的两条对角线将数组分为了上下左右四个部分，黄色部分为对角线，绿色部分为左方区域。

输入

第一行输入一个大写字母，若为S，则表示需要求出左方区域的元素的和，若为M，则表示需要求出左方区域的元素的平均值。接下来 12 行，每行包含 12 个用空格隔开的浮点数，表示这个二维数组，其中第 $i + 1$ 行的第 $j + 1$ 个数表示数组元素 $M[i][j]$ 。数据范围 $-100.0 \leq M[i][j] \leq 100.0$

输出

输出一个数，表示所求的平均数或和的值，保留一位小数。

来源

AcWing 751

输入

```
S
4.5 -3.3 -2.3 4.5 -7.0 8.7 -4.1 -3.0
-7.6 6.3 -6.6 -4.7
4.7 -9.3 -7.6 9.1 9.2 9.0 5.5 -7.5 -9.3
-1.6 -3.5 -4.2
0.5 -7.5 -8.3 -9.0 -9.6 4.3 8.0 -1.5
7.9 2.1 2.4 -6.2
-4.4 -1.3 -9.7 -3.4 -7.0 8.4 -8.5 -9.3
-1.4 -2.4 -5.1 -7.0
3.8 -9.9 6.2 -2.5 -3.5 9.4 1.6 7.0 3.3
-0.5 6.7 6.0
1.6 -3.8 5.0 8.8 4.2 7.7 0.7 7.4 7.9
```

输出

```
13.3
```

```

-5.9 4.4 3.3
3.7 6.0 2.6 -1.4 6.1 -6.0 8.5 9.1 5.7
-4.2 5.9 -3.5
5.0 0.3 2.2 -3.6 6.3 -10.0 9.5 -7.7
-8.2 -4.3 -5.3 -3.9
-7.4 -2.9 -3.3 -7.7 -2.5 -5.5 -3.1 1.0
3.1 0.9 9.9 0.0
-7.4 1.3 -9.6 -3.6 2.2 2.3 -4.3 -3.6
6.5 8.3 0.5 7.6 -4.8
1.8 -2.7 -1.5 5.4 -6.5 -3.7 5.6 8.0
-9.9 0.1 2.2 7.6
5.6 4.3 1.5 -0.8 5.8 -5.1 5.5 6.2 -5.8
8.8 0.0 -6.2 -2.3

```

提示 注意输出格式。

C++

```

#include <iostream>
#include <cstdio>

using namespace std;

int main()
{
    char t;
    cin >> t;

    double m[12][12];
    for (int i = 0; i < 12; i++)
        for (int j = 0; j < 12; j++)
            cin >> m[i][j];

    double s = 0, c = 0;
    for (int i = 1; i <= 5; i++)
        for (int j = 0; j <= i - 1; j++)
        {
            s += m[i][j];
            c += 1;
        }
    for (int i = 6; i <= 10; i++)
        for (int j = 0; j <= 10 - i; j++)
        {
            s += m[i][j];
            c += 1;
        }
    if (t == 'M') printf("%.1lf\n", s / c);
    else printf("%.1lf\n", s);

    return 0;
}

```

Python

```

# AcWing 751
# Python solution

```

NQ062 平方矩阵 I AcWing 753

输入整数 N ，输出一个 N 阶的回字形二维数组。数组的最外层为 1，次外层为 2，以此类推。

输入

输入包含多行，每行包含一个整数 N 。当输入行为 $N = 0$ 时，表示输入结束，且该行无需作任何处理。数据范围 $0 \leq N \leq 100$

输出

对于每个输入整数 N ，输出一个满足要求的 N 阶二维数组。每个数组占 N 行，每行包含 N 个用空格隔开的整数。每个数组输出完毕后，输出一个空行。

来源

AcWing 753

输入	输出
1	1
2	
3	1 1
4	1 1
0	
	1 1 1
	1 2 1
	1 1 1
	1 1 1 1
	1 2 2 1
	1 2 2 1
	1 1 1 1

提示 注意输出格式。

C++

```
#include <iostream>

using namespace std;

int main()
{
    int n;
    while (cin >> n, n)
    {
        for (int i = 1; i <= n; i++)
        {
            for (int j = 1; j <= n; j++)
            {
                int up = i, down = n - i + 1, left = j, right =
n - j + 1;
                cout << min(min(up, down), min(left, right)) <<
' ';
            }
            cout << endl;
        }

        cout << endl;
    }

    return 0;
}
```

Python

```
# AcWing 753
# Python solution
```

输入一个二维数组 $M[12][12]$ ，根据要求，求出二维数组的右下半部分元素的平均值或元素的和。右下半部分是指次对角线下方的部分，黄色部分为对角线，绿色部分为右下半部分。

输入

第一行输入一个大写字母，若为 S，则表示需要求出右下半部分的元素的和，若为 M，则表示需要求出右下半部分的元素的平均值。接下来 12 行，每行包含 12 个用空格隔开的浮点数，表示这个二维数组。

输出

输出一个数，表示所求的平均值或和的值，保留一位小数。

来源

AcWing 748

输入

```
S
-4.9 0.1 -6.1 -9.6 1.0 -3.2 0.6 3.2
-9.8 4.9 1.2
9.7 -8.5 3.2 8.8 -1.9 -5.4 7.5 -2.0 5.7
2.3 5.3 -7.5 0.9
-6.8 -4.3 3.8 -6.7 5.6 -8.2 -7.1 -7.9
-3.7 -7.4 -4.4 -5.3
6.8 4.3 3.8 -7.7 -1.4 -1.8 -1.5 -1.2
-5.4 7.8 2.1 4.8 4.7
-7.3 -1.7 -4.7 -3.1 -1.4 3.1 -2.5 -4.7
8.2 -1.4 -1.8 -4.7
2.4 9.4 -4.3 9.6 8.6 -6.1 -7.4 8.6 0.5
-8.4 5.2
-5.2 2.9 -5.6 4.0 -8.2 3.8 -4.1 -1.6
-3.8 -3.1 -1.1 -3.3
-3.3 -3.6 -4.3 -5.4 -2.5 -4.3 -7.1 -2.5
-3.1 -1.8 -7.4 -4.5
-2.9 -3.6 7.1 -9.5 -8.7 8.2 -6.2 -1.0
7.4 -3.0 -4.4 -4.5
0.1 9.5 4.9 1.5 0.8 -8.2 -0.4 9.5 -7.0
-1.0 -9.7 -2.1
-0.1 -7.6 7.8 -0.5 -6.5 4.4 -0.4 -4.6
-7.8 -0.6 -0.1 -1.9
-2.3 -8.4 -2.7 -3.4 -3.9 -7.1 -4.8 -1.3
-5.4 -3.3 -2.4 -3.0
-0.1 -8.2 0.8 -3.5 -7.6 -0.5 5.6 8.4
-0.6 0.9 9.9 -7.5
```

输出

```
53.0
```

提示 先判断第一行字母，再计算右下半部分数据的和或平均值，保留一位小数。来源：语法题

C++

```
#include <cstdio>
#include <iostream>

using namespace std;

int main()
{
    char t;
    cin >> t;
    double m[12][12];

    for (int i = 0; i < 12; i ++ )
        for (int j = 0; j < 12; j ++ )
            cin >> m[i][j];

    double s = 0, c = 0;
    for (int i = 1; i <= 11; i ++ )
        for (int j = 12 - i; j <= 11; j ++ )
        {
            s += m[i][j];
            c += 1;
        }

    if (t == 'S') printf("%.1lf\n", s);
    else printf("%.1lf\n", s / c);

    return 0;
}
```

Python

```
# AcWing 748
# Python solution
```

NQ064 数组的左下半部分 AcWing 746

输入一个二维数组 $M[12][12]$ ，根据要求，求出二维数组的左下半部分元素的平均值或元素的和。左下半部分是指主对角线下方的部分，黄色部分为对角线，绿色部分为左下半部分。

输入 第一行输入一个大写字母，若为 S，则表示需要求出左下半部分的元素的和，若为 M，则表示需要求出左下半部分的元素的平均值。接下来 12 行，每行包含 12 个用空格隔开的浮点数，表示这个二维数组。

输出 输出一个数，表示所求的平均值或和的值，保留一位小数。

来源 AcWing 746

输入

```
S
8.7 5.6 -2.0 -2.1 -7.9 -9.0 -6.4 1.7
2.9 -2.3 -8.4 0.0
-7.3 -2.1 0.6 -9.8 9.6 5.6 -1.3 -3.8
-9.3 -8.0 -2.0 2.9
-4.9 -1.5 -5.5 -5.2 -4.5 -4.1 7.6 6.9
-9.0 6.8 9.1 -8.5
-2.5 -2.0 -1.8 -5.9 -7.2 -3.3 -1.8 -4.3
-5.5 -7.1 -7.7 -3.7
-7.0 -1.2 7.8 7.1 -3.7 7.8 -2.3 4.3 0.2
-4.6 -6.6
-3.2 -3.4 -4.7 4.7 3.6 8.8 5.1 -3.2
```

输出

```
-2.8
```

```

-1.9 2.8 -4.3 -1.4
-3.0 -2.0 -7.1 -1.3 -3.7 -1.3 -1.4 -3.4
-4.0 -2.7 -4.4 -3.4
-2.5 -0.9 9.2 -8.4 -9.4 -5.1 6.5 1.5
3.4 -0.1 -7.0 -6.8
-1.8 -4.9 2.2 -8.7 -9.4 4.8 2.3 0.2
-3.2 -7.5 -4.0 -9.9
-1.1 -2.9 0.7 -0.7 -7.4 8.6 -2.0 -1.7
-0.7 -0.8 -5.4 -3.8
-5.4 -8.7 -7.1 -5.7 -1.4 -1.8 -3.3 -4.1
-2.7 -2.9 -4.6 -5.5
-2.8 -9.1 7.1 -0.2 -8.6 -6.5 1.1 -0.1
-3.6 4.0 -5.4 1.1

```

提示 先判断第一行字母，再计算左下半部分数据的和或平均值，保留一位小数。来源：语法题

C++

```

#include <cstdio>
#include <iostream>

using namespace std;

int main()
{
    char t;
    cin >> t;

    double m[12][12];
    for (int i = 0; i < 12; i++)
        for (int j = 0; j < 12; j++)
            cin >> m[i][j];

    double s = 0, c = 0;
    for (int i = 0; i < 12; i++)
        for (int j = 0; j <= i - 1; j++)
        {
            s += m[i][j];
            c++;
        }

    if (t == 'S') printf("%.1lf\n", s);
    else printf("%.1lf\n", s / c);

    return 0;
}

```

Python

```

# AcWing 746
# Python solution

```

NQ065 数组的下方区域 AcWing 750

输入一个二维数组 $M[12][12]$ ，根据要求，求出二维数组的下方区域元素的平均值或元素的和。数组的两条对角线将数组分为了上下左右四个部分，黄色部分为对角线，绿色部分为下方区域。

输入

第一行输入一个大写字母，若为 S，则表示需要求出下方区域的元素的和，若为 M，则表示需要求出下方区域的元素的平均值。接下来 12 行，每行包含 12 个用空格隔开的浮点数，表示这个二维数组。

输出

输出一个数，表示所求的平均值或和的值，保留一位小数。

来源

AcWing 750

输入

```

S
-0.7 -8.4 -5.7 -1.1 7.6 9.5 -9.7 4.1
0.6 -6.5 -4.9
-6.6 4.9 -3.1 5.3 0.3 -4.5 3.9 -1.5 6.7
0.5 1.2 5.5
-8.5 1.8 -7.7 0.2 -9.7 -7.2 4.3 8.8
-7.9 -9.0 -2.7 -8.8
-4.0 -9.3 -4.2 -3.7 3.3 -2.4 -3.8 1.0
-3.0 -9.9 -8.5 8.8
-2.1 -9.6 5.0 2.1 -8.7 6.4 4.2 -8.9 2.4
9.3
-6.9 -0.1 -7.8 -7.0 7.8 5.1 6.9 -7.6
0.4 -7.2 5.5 1.8
-8.6 -7.7 -7.0 -3.4 -1.7 -9.1 7.4 -9.4
-4.7 -4.4 -2.6 -0.1
0.2 2.3 3.5 1.4 0.5 -1.1 -4.9 2.8 -1.7
-0.7 0.8 -6.0
-6.0 -4.1 -4.6 -9.4 -4.9 -4.1 4.7 6.3
-7.8 0.7 -8.1 -0.9
8.8 -4.5 -6.6 0.6 -4.0 -7.2 0.3 -2.8
0.7 -0.1 -4.9
-3.8 -7.4 -4.9 -0.2 -4.3 -9.3 -0.4 -2.4
-8.7 -2.7 -4.1 -3.7
-3.8 -6.4 2.3 4.2 -9.8 -0.3 -9.9 -7.4
3.5 1.5 -0.2 7.8

```

输出

-11.9

提示 先判断第一行字母，再计算下方区域数据的和或平均值，保留一位小数。来源：语法题

C++

```
#include <iostream>
#include <cstdio>

using namespace std;

int main()
{
    char t;
    cin >> t;

    double m[12][12];
    for (int i = 0; i < 12; i++)
        for (int j = 0; j < 12; j++)
            cin >> m[i][j];

    double s = 0, c = 0;
    for (int i = 7; i <= 11; i++)
        for (int j = 12 - i; j <= i - 1; j++)
        {
            c += 1;
            s += m[i][j];
        }
    if (t == 'M') printf("%.1lf\n", s / c);
    else printf("%.1lf\n", s);

    return 0;
}
```

Python

```
# AcWing 750
# Python solution
```

NQ066 数组的右方区域 AcWing 752

输入一个二维数组 $M[12][12]$ ，根据要求，求出二维数组的右方区域元素的平均值或元素的和。数组的两条对角线将数组分为了上下左右四个部分，黄色部分为对角线，绿色部分为右方区域。

输入 第一行输入一个大写字母，若为 S，则表示需要求出右方区域的元素的和，若为 M，则表示需要求出右方区域的元素的平均值。接下来 12 行，每行包含 12 个用空格隔开的浮点数，表示这个二维数组。

输出 输出一个数，表示所求的平均值或和的值，保留一位小数。

来源 AcWing 752

输入

```
S
2.4 7.8 9.4 -5.6 6.9 -4.9 4.8 0.8 3.6
1.7 -1.4 9.7
-6.8 -3.7 -2.0 -4.9 -4.5 -3.3 6.1 7.5
-4.3 5.9 -9.5 9.7
-5.5 -4.1 4.6 3.7 -4.4 -3.3 1.9 7.7
-4.7 -3.9 -7.1 -5.0
-3.4 -4.9 -7.3 -7.4 -4.0 -7.3 -3.8 -1.6
-4.8 8.5 3.9
-0.2 -2.0 -6.4 -9.8 3.7 -2.0 1.7 -3.6
-3.4 2.4 -1.2 -3.9
-9.3 3.8 -1.8 -4.4 -1.0 -2.4 2.8 -4.6
2.1 8.7 -6.8 -8.3
-4.4 -4.7 -2.3 -7.7 -3.7 -4.4 -1.2 -2.7
```

输出

```
40.9
```

```

-4.8 -1.3 -7.1 -2.1
0.4 8.6 -1.2 8.6 -1.2 8.9 1.6 2.8 -1.0
-3.9 2.7 -3.4
7.5 -3.1 6.2 -4.5 -3.0 -2.5 -7.7 2.9
0.3 -3.3 -2.7 3.4
-5.8 -0.6 -0.4 -5.6 -0.6 -0.9 -0.7 -0.8
0.3 -3.7 -7.4 6.5
-3.3 -5.3 -7.3 -7.3 -0.8 -1.6 -1.3 -1.9
-1.3 -1.7 -2.1 -2.7
-6.4 6.8 -3.7 -4.7 0.2 5.8 -5.4 0.6 7.0
-4.2 -7.5 -2.4

```

提示 先判断第一行字母，再计算右方区域数据的和或平均值，保留一位小数。来源：语法题

C++

```

#include <iostream>
#include <cstdio>

using namespace std;

int main()
{
    char t;
    cin >> t;
    double m[12][12];

    for (int i = 0; i < 12; i++)
        for (int j = 0; j < 12; j++)
            cin >> m[i][j];

    double s = 0, c = 0;
    for (int i = 1; i <= 5; i++)
        for (int j = 12 - i; j <= 11; j++)
        {
            s += m[i][j];
            c += 1;
        }
    for (int i = 6; i <= 10; i++)
        for (int j = i + 1; j <= 11; j++)
        {
            s += m[i][j];
            c += 1;
        }
    if (t == 'M') printf("%.1lf\n", s / c);
    else printf("%.1lf\n", s);

    return 0;
}

```

Python

```

# AcWing 752
# Python solution

```

NQ067 平方矩阵 II AcWing 754

输入整数 N ，输出一个 N 阶的二维数组。数组的形式参照样例。

输入

输入包含多行，每行包含一个整数 N 。当输入行为 $N = 0$ 时，表示输入结束，且该行无需作任何处理。数据范围 $0 \leq N \leq 100$

输出

对于每个输入整数 N ，输出一个满足要求的 N 阶二维数组。每个数组占 N 行，每行包含 N 个用空格隔开的整数。每个数组输出完毕后，输出一个空行。

来源

AcWing 754

输入	输出
1	1
2	
3	1 2
4	2 1
5	
0	1 2 3
	2 1 2
	3 2 1
	1 2 3 4
	2 1 2 3
	3 2 1 2
	4 3 2 1
	1 2 3 4 5
	2 1 2 3 4
	3 2 1 2 3
	4 3 2 1 2
	5 4 3 2 1

提示 注意输出格式。

C++

```
#include <iostream>

using namespace std;

int q[100][100];

int main()
{
    int n;
    while (cin >> n, n)
    {
        for (int i = 0; i < n; i++)
        {
            q[i][i] = 1;
            for (int j = i + 1, k = 2; j < n; j++, k++) q[i][j] = k;
            for (int j = i + 1, k = 2; j < n; j++, k++) q[j][i] = k;
        }

        for (int i = 0; i < n; i++)
        {
            for (int j = 0; j < n; j++) cout << q[i][j] << ' ';
            cout << endl;
        }
        cout << endl;
    }

    return 0;
}
```

Python

```
# AcWing 754
# Python solution
```

NQ068 平方矩阵 III AcWing 755

输入整数 N ，输出一个 N 阶的二维数组 M ，这个 N 阶二维数组满足 $M[i][j] = 2^{(i+j)}$ 。具体形式可参考样例。

输入

输入包含多行，每行包含一个整数 N 。当输入行为 $N = 0$ 时，表示输入结束，且该行无需作任何处理。
数据范围 $0 \leq N \leq 15$

输出

对于每个输入整数 N ，输出一个满足要求的 N 阶二维数组。每个数组占 N 行，每行包含 N 个用空格隔开的整数，输出完毕后需再输出一个空行。

来源

AcWing 755

输入	输出
1	1
2	
3	1 2
4	2 4
5	
0	1 2 4
	2 4 8
	4 8 16

```

1 2 4 8
2 4 8 16
4 8 16 32
8 16 32 64

1 2 4 8 16
2 4 8 16 32
4 8 16 32 64
8 16 32 64 128
16 32 64 128 256

```

提示 注意输出格式。

C++

```

#include <iostream>
#include <cstdio>

using namespace std;

int main()
{
    int n;
    while (cin >> n, n)
    {
        for (int i = 0; i < n; i++)
        {
            for (int j = 0; j < n; j++)
            {
                int v = 1;
                for (int k = 0; k < i + j; k++) v *= 2;
                cout << v << ' ';
            }
            cout << endl;
        }

        cout << endl;
    }

    return 0;
}

```

Python

```

# AcWing 755
# Python solution

```

NQ069 蛇形矩阵 AcWing 756

输入两个整数 n 和 m ，输出一个 n 行 m 列的矩阵，将数字 1 到 $n \times m$ 按照回字蛇形填充至矩阵中。具体蛇形形式可参考样例。

输入 输入共一行，包含两个整数 n 和 m 。

输出 输出满足要求的矩阵。矩阵占 n 行，每行包含 m 个用空格隔开的整数，输出完毕后需再输出一个空行。

来源 AcWing 756

输入

输出

3 3

```

1 2 3
8 9 4
7 6 5

```

C++

```

#include <iostream>

using namespace std;

int res[100][100];

int main()
{
    int n, m;
    cin >> n >> m;

    int dx[] = {0, 1, 0, -1}, dy[] = {1, 0, -1, 0};

    for (int x = 0, y = 0, d = 0, k = 1; k <= n * m; k++)
    {
        res[x][y] = k;
        int a = x + dx[d], b = y + dy[d];
        if (a < 0 || a >= n || b < 0 || b >= m || res[a]
[b])
        {
            d = (d + 1) % 4;
            a = x + dx[d], b = y + dy[d];
        }
        x = a; y = b;
    }

    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < m; j++) cout << res[i][j] <<
' ';
        cout << endl;
    }

    return 0;
}

```

Python

```

# AcWing 756
# Python solution

```